

POLITECNICO
MILANO 1863

054323 - INTERNET OF THINGS

POLITECNICO DI MILANO

COMPUTER SCIENCE AND ENGINEERING

IoT Challenge 2
Part 1

Authors:
Carmen Giaccotto (Person Code: ********)
Alessia Franchetti-Rosada (Person Code: ********)

Date: April 23, 2026

1 Questions on the PCAP file ”A.pcapng”

1.1 CQ1

1.1.1 CQ1a - Analysis of DELETE Requests

Filter used in Wireshark

```
coap && ip.dst == 134.102.218.18 && coap.type == 1 && coap.code == 0.04
```

This filter returns all 20 NON Confirmable DELETE requests directed to the `coap.me` server (`134.102.218.18`). The full list is reported below:

#	MID	Token	URI
1	33264	`7b 93 76 a6 a9 48 4b 8b`	`/5`
2	53878	`03 0b da 63 ff 31 59 2a`	`/broken`
3	63041	`f3 5b bf 03 a5 ff 38 cc`	`/3`
4	38052	`23 01 e4 de 41 56 74 9f`	`/weird333`
5	57936	`e9 3d bb cb 6a 59 8a 7d`	`/location1`
6	25814	`8b 6f 78 cb e0 76 f9 16`	`/hello`
7	1059	`b6 51 a7 4c 2a 74 ae cd`	`/separate`
8	32298	`73 59 ba 94 93 b3 9e 78`	`/4`
9	24716	`e0 86 49 98 dd 73 82 06`	`/seg1`
10	38480	`57 f9 87 c1 79 a3 2d 78`	`/secret`
11	32242	`78 01 af 5e a0 b3 48 f0`	`/seg1`
12	30212	`25 2e f4 36 b7 ef a3 ea`	`/path`
13	11787	`21 a5 87 c5 a0 ae 8a 6e`	`/3`
14	22836	`7f 09 e3 67 7e bb cb 42`	`/4`
15	63788	`07 f2 0b 6a d0 47 d7 49`	`/5`
16	47220	`1e 60 83 b6 96 fb 32 31`	`/3`
17	62535	`73 70 51 f7 9b a1 e4 d2`	`/seg1`
18	49932	`45 2a 18 97 9a 58 70 44`	`/secret`
19	30800	`88 0f 30 d8 b5 43 d6 f7`	`/validate`
20	21028	`75 91 2d 3b 51 bc 50 16`	`/3`

Table 1: NON Confirmable DELETE requests directed to `coap.me`

To identify which of these received a successful response, the responses from `coap.me` were inspected using the filter:

```
coap && ip.src == 134.102.218.18
```

For each response, the token was matched against the 20 requests above. A response is considered successful if its code is in the 2.xx class (2.01 Created, 2.02 Deleted, 2.03 Valid, 2.04 Changed, 2.05 Content).
Only one request received a successful response:

MID	Token	URI	Response code
30800	`88 0f 30 d8 b5 43 d6 f7`	`/validate`	2.02 Deleted

Table 2: NON DELETE request that received a successful response

Answer CQ1a

CQ1a: MID = 30800

1.1.2 CQ1b - Verification of Desired Outcome

CQ1b asks how many of the NON DELETE requests identified in CQ1a actually obtained the desired outcome, defined as the resource being verifiably absent after the DELETE operation.
From CQ1a, only one NON DELETE request received a successful response from `coap.me`: the DELETE to resource `/validate` with MID 30800, which returned a 2.02 Deleted response.
To verify whether the resource was actually deleted, the capture was inspected for any subsequent GET request to the same resource `/validate` on `coap.me` after the DELETE operation. The following Wireshark filter was applied:

```
coap && ip.dst == 134.102.218.18 && coap.opt.uri_path == "validate"
```

A subsequent GET request to `/validate` was found at packets 11363–11364, sent after the DELETE with MID 30800. The response returned 2.05 Content, meaning the server responded with the resource content, confirming that the resource `/validate` was still present on `coap.me` after the DELETE.
This behavior is consistent with `coap.me` being a public test server that does not perform persistent deletions. Although the server acknowledged the DELETE with a 2.02 Deleted response, the resource was not actually removed.
Therefore, the desired outcome was not achieved: the resource still existed after the DELETE operation.

Answer CQ1b

CQ1b = 0
No NON DELETE request to `coap.me` actually obtained the desired outcome. The only candidate (MID 30800, resource `/validate`) received a 2.02 Deleted response, but a subsequent GET to the same resource returned 2.05 Content, proving that the resource was not deleted.

1.2 CQ2 - Unsuccessful CON POST and CON PUT Requests

1.2.1 Identify the local server

The local CoAP server is identified by IP address `127.0.0.1` on UDP port 5683, as opposed to the external `coap.me` server (`134.102.218.18`).

1.2.2 Filter CON POST and CON PUT requests to the local server

The following Wireshark filters were applied to isolate the relevant requests.
For CON POST requests:

```
coap && ip.dst == 127.0.0.1 && coap.type == 0 && coap.code == 0.02
```

For CON PUT requests:

```
coap && ip.dst == 127.0.0.1 && coap.type == 0 && coap.code == 0.03
```

This returned 22 CON POST requests and 11 CON PUT requests directed to the local server.

1.2.3 Identify unsuccessful requests

A request is considered unsuccessful only if a response explicitly stating failure is present in the capture, i.e., a response with code class 4.xx (Client Error) or 5.xx (Server Error). Requests with no response at all are not counted.
For CON messages, the response is matched via MID: the server replies with an ACK carrying the same MID as the request. Two sub-cases exist:

- **Piggybacked ACK:** the ACK directly contains the response code (code $\neq$ 0.00)
- **Empty ACK + separate response:** the server first sends an ACK with code 0.00 (Empty), followed by a separate response matched via token

To verify each request on Wireshark, the following filter was used after identifying the MID of each request:

```
coap && ip.src == 127.0.0.1 && coap.type == 2 && coap.mid == <MID>
```

1.2.4 Count unique unsuccessful requests per resource

For each resource (URI path), the number of distinct MIDs of failed CON POST (X) and failed CON PUT (Y) requests was counted. The results are reported below.

Unsuccessful CON POST requests per resource:

Resource	**Failed MIDs**	X
`/test`	64080, 6295	2
`/dining_room/temperature`	10465, 10466, 53552, 11409, 21336	5
`/hello_world`	57914, 25749, 29078, 26063	4
`/dining_room`	40958	1
`/basic`	11242, 42780	2
`/main_door`	33536	1
`/living_room/temperature`	21082	1
`/living_room`	40534	1
`/dining_room/door`	10179, 9853	2

Table 3: Unsuccessful CON POST requests per resource

Unsuccessful CON PUT requests per resource:

Resource	**Failed MIDs**	Y
`/main_door`	1330, 30514	2
`/dining_room/temperature`	48174, 40207	2
`/basic`	50477	1
`/dining_room/door`	33760	1
`/living_room/door`	6326	1
`/dining_room`	10680	1

Table 4: Unsuccessful CON PUT requests per resource

Comparison table ($X = Y$ with $X > 0$):

Resource	X (failed POST)	Y (failed PUT)	$X = Y$?
/test	2	0	×
/dining_room/temperature	5	2	×
/hello_world	4	0	×
/dining_room	1	1	✓
/basic	2	1	×
/main_door	1	2	×
/living_room/temperature	1	0	×
/living_room	1	0	×
/dining_room/door	2	1	×
/living_room/door	0	1	×

Table 5: Comparison of failed POST and PUT counts per resource

Only resource /dining_room satisfies the condition $X = Y$ with $X > 0$, having exactly 1 unique unsuccessful CON POST (MID: 40958) and 1 unique unsuccessful CON PUT (MID: 10680), both confirmed by explicit error responses from the local server.

Answer CQ2

CQ2 = 1
The only CoAP resource in the local server that received the same number of unique unsuccessful Confirmable POST and PUT requests is /dining_room, with $X = Y = 1$.

1.3 CQ3

1.3.1 CQ3a - Separate Observe Notifications

Identify the Observe registration

The CoAP Observe mechanism works as follows: the client sends a GET request with the Observe option set to 0 to register as an observer of a resource. The server then sends notifications autonomously every time the resource value changes, using the same token as the original GET request. Each notification carries an incrementing Observe sequence number.
The following Wireshark filter was applied to find the registration for /dining_room/temperature:

```
coap && coap.opt.observe == 0 && coap.opt.uri_path == "temperature"
```

One single Observe registration was found:

Field	Value
Type	CON
Code	GET
MID	10588
Token	89b9
URI	/dining_room/temperature

Table 6: Observe registration for /dining_room/temperature

The token 89b9 is the key linking all subsequent notifications to this registration.

Retrieve all notifications

The following filter was applied to retrieve all messages associated with this Observe registration:

```
coap && coap.token == 89b9 && ip.src == 127.0.0.1
```

Only messages with the Observe option present and a 2.xx response code were considered as actual notifications. A total of 15 raw messages were found.

Remove retransmissions and identify separate notifications

The question asks specifically for *separate* observe notifications, ignoring retransmissions:

- **Piggybacked response:** included directly inside the ACK of the original CON request — this is *not* a separate notification.
- **Separate notification:** sent autonomously, independently of any ACK.

The first message in the list (MID 10588, ACK, Observe seq. 1, value 19.97) is a piggybacked response and is therefore excluded from the count.
All remaining CON notifications are separate notifications sent autonomously by the server. However, some are retransmissions (identified by repeated MIDs). Duplicate MIDs are removed and only the first occurrence of each MID is kept:

- MID 54 appears 4 times → 1 original + 3 retransmissions removed
- MID 62 appears 2 times → 1 original + 1 retransmission removed

After removing retransmissions and excluding the piggybacked response, the unique separate notifications are:

MID	Type	Observe seq.	Temperature value
35	CON	1	22.32
38	CON	2	18.85
43	CON	3	25.98
45	CON	4	15.59
49	CON	5	25.35
54	CON	6	26.21
62	CON	7	27.53
74	CON	8	26.98
87	CON	9	25.32
90	CON	10	19.57

Table 7: Unique separate Observe notifications for `/dining_room/temperature`

Answer CQ3a

CQ3a = 10
There are 10 unique separate observe notifications for resource `/dining_room/temperature`.

1.3.2 CQ3b - Useless Notifications (Traffic Waste)

A notification is considered useless (a waste of traffic) when it conveys no useful information to the client. Two different levels of analysis were performed.

Analysis 1 - Application level (payload comparison)

At the application level, a notification is useless if the temperature value it carries is identical to the value of the previous notification. The payload of each unique notification was inspected in Wireshark:

Observe seq.	MID	Temperature value	Same as previous?
1	35	22.32	— (first separate)
2	38	18.85	No
3	43	25.98	No
4	45	15.59	No
5	49	25.35	No
6	54	26.21	No
7	62	27.53	No
8	74	26.98	No
9	87	25.32	No
10	90	19.57	No

Table 8: Payload comparison between consecutive Observe notifications

All temperature values are distinct. From a pure application-level perspective, no notification is redundant (**0 useless notifications**).

Analysis 2 - Network level (RFC 7641 compliance)

At the network level, a deeper analysis reveals a different picture. Starting from notification MID 54 (Observe seq. 6), the server consistently receives ICMP Destination Unreachable (Port Unreachable) messages from the client side, verifiable with:

```
icmp && icmp.type == 3 && icmp.code == 3
```

These ICMP messages indicate that the client UDP port is closed. According to RFC 7641 Section 4.5, when a server receives an ICMP Port Unreachable error in response to a notification, it **MUST** remove the observer from its list and stop sending further notifications.

However, the server violates this requirement and continues to send 5 notifications (MID 54, 62, 74, 87, 90) towards a closed port. These are a pure waste of traffic:

MID	Observe seq.	Temp.	Reason useless
54	6	26.21	Sent after ICMP Port Unreachable received
62	7	27.53	Sent after ICMP Port Unreachable received
74	8	26.98	Sent after ICMP Port Unreachable received
87	9	25.32	Sent after ICMP Port Unreachable received
90	10	19.57	Sent after ICMP Port Unreachable received

Table 9: Useless notifications sent after ICMP Port Unreachable

Summary

Analysis level	Useless	Reasoning
Application level	0	All payload values are distinct
Network level (RFC 7641)	5	Server ignores ICMP and keeps sending to a closed port

Table 10: Summary of useless notification analysis

Answer CQ3b

CQ3b = 5
There are 5 useless notifications: starting from MID 54, the server receives ICMP Destination Unreachable messages but violates RFC 7641 by continuing to send 5 further notifications towards a closed port, generating pure traffic waste.

1.4 CQ4 - Analysis of MQTT-SN Traffic

1.4.1 Configure Wireshark for MQTT-SN on port 1885

Since Wireshark does not automatically dissect MQTT-SN traffic on this specific port by default, the MQTT-SN protocol dissector was manually configured to decode traffic on UDP port 1885.

1.4.2 Apply the MQTT-SN filter

The following filter was applied in the Wireshark display filter bar:

`mqttsn`

This returns all packets that Wireshark recognizes and dissects as MQTT-SN traffic on port 1885.

1.4.3 Results

The filtered output reveals a total of 219 MQTT-SN packets, all of type `Publish Message`, all exchanged between `127.0.0.1` and `127.0.0.1` (loopback interface).
A critical observation is immediately apparent from the capture: every single MQTT-SN Publish Message is followed immediately by an ICMP Destination Unreachable (Port Unreachable) response, as shown in the pattern below:

Packet	Timestamp	Protocol	Info
917	78.195846	MQTT-SN	Publish Message
918	78.195854	ICMP	Destination unreachable (Port unreachable)
984	79.377493	MQTT-SN	Publish Message
985	79.377515	ICMP	Destination unreachable (Port unreachable)
1009	79.710743	MQTT-SN	Publish Message
1010	79.710784	ICMP	Destination unreachable (Port unreachable)
			⋮

Table 11: Repeating pattern of MQTT-SN Publish followed by ICMP Port Unreachable

This pattern repeats consistently for all 219 MQTT-SN packets throughout the entire capture.

1.4.4 Interpretation

The ICMP Destination Unreachable (Port Unreachable) message is generated by the operating system when a UDP packet is sent to a port on which no process is listening. The presence of this ICMP response after every single MQTT-SN Publish Message indicates that:

- The clients are sending Publish Messages to port 1885 on the loopback address.
- No broker is actively listening on port 1885 during the entire capture.
- Every message is immediately rejected by the OS before it can be processed.
- Since the broker is not running, no message is ever delivered to it.
- Consequently, no client can receive any message from a broker that does not exist.

Additionally, all 219 MQTT-SN Publish Messages use **QoS = −1** (value 3 in the flags byte), which in MQTT-SN means “fire and forget without a connection”. This QoS level requires no acknowledgement from the broker, which is consistent with the absence of any broker response in the capture.

Answer CQ4

CQ4 = 0
No MQTT-SN messages were received by any client from the local broker listening on port 1885. The broker was not running during the capture period, as evidenced by the ICMP Destination Unreachable (Port Unreachable) response that immediately follows every single one of the 219 MQTT-SN Publish Messages present in the file.

2 Questions on the PCAP file ”B.pcapng”

2.1 CQ5

2.1.1 Last Will and Testament mechanism

In MQTT, the **Last Will and Testament (LWT)** mechanism works as follows:

- A client connects to a broker including a **Will Topic** and a **Will Message** in its `CONNECT` packet.
- If that client disconnects **abnormally** (without sending a `DISCONNECT` message), the broker automatically publishes the Will Message on the Will Topic.
- All clients with an active subscription matching the Will Topic will receive the message.

2.1.2 Identify the brokers in B.pcapng

By applying the filter `mqtt.msgtype == 1` and inspecting the `CONNECT` packets, the following brokers were identified:

IP	Type
`::1`	Local broker (IPv6 loopback)
`18.192.151.104`	HiveMQ public broker
`5.196.78.28`	External broker
`35.158.43.69`	HiveMQ public broker

Table 12: Brokers identified in B.pcapng

2.1.3 Find all clients with a Last Will

The following Wireshark filter was applied to find all `CONNECT` messages with the Will flag set:

```
mqtt.conflag.willflag == 1
```

Four clients with a Last Will were found:

Packet	Broker	Will Topic
4	`::1`	`university/department12/room1/temperature`
196	`5.196.78.28`	`metaverse/room2/floor4`
352	`5.196.78.28`	`hospital/facility3/area3`
557	`5.196.78.28`	`metaverse/room2/room2`

Table 13: Clients with a Last Will in B.pcapng

2.1.4 Find all wildcard subscriptions

The following Wireshark filter was applied to find all `SUBSCRIBE` messages:

```
mqtt.msgtype == 8
```

From the results, all `SUBSCRIBE` messages directed to external brokers (`18.192.151.104`) were found, as well as subscriptions to the local broker `::1`. Only subscriptions containing at least one wildcard (`+` or `#`) are relevant for matching.

2.1.5 Match Will Topics with wildcard subscriptions on the same broker

A subscriber can only receive a Will Message if it is connected to the **same broker** as the client that has the Will. Therefore, for each Will Topic, only subscriptions on the same broker were considered.
Broker `::1` is the only broker that has both:

- A client with a Will (packet 4, Will Topic: `university/department12/room1/temperature`).
- Subscribers with wildcard subscriptions.

The Will Topic to match is `university/department12/room1/temperature`, which has 4 levels:

- 1 `university`
- 2 `department12`

- 3 `room1`
- 4 `temperature`

The MQTT wildcard matching rules are:

- `+` matches exactly one topic level.
- `#` matches all remaining levels and must be the last character.

The following subscriptions on `::1` were checked against the Will Topic:

Topic	Match?	Reason
`university/+/+/deposit`	×	Level 4 = `deposit` ≠ `temperature`
`university/facility0/#`	×	Level 2 = `facility0` ≠ `department12`
`university/department3/#`	×	Level 2 = `department3` ≠ `department12`
`university/department3/+`	×	Level 2 = `department3` ≠ `department12`
`university/room0/#`	×	Level 2 = `room0` ≠ `department12`
`university/building2/section0/#`	×	Level 2 = `building2` ≠ `department12`
`university/#`	✓	Level 1 = `university` ✓

Table 14: Wildcard subscription matching against the Will Topic on broker `::1`

Only one subscription matches the Will Topic: `university/#` found at packet 1136, directed to broker `::1`.
Broker `5.196.78.28` has three clients with Wills but no subscribers. Therefore none of the Will Messages on `5.196.78.28` can be received by any subscriber.

Answer CQ5

CQ5 = 1
Only one subscriber will receive a Last Will message via a wildcard subscription. This is the client subscribed to `university/#` on the local broker `::1` (packet 1136), whose subscription matches the Will Topic `university/department12/room1/temperature` registered by the client connecting at packet 4 on the same broker.

2.2 CQ6

2.2.1 CQ6a - Identify the HiveMQ public broker

The HiveMQ public broker was identified by applying the following DNS filter to find which IP addresses correspond to `broker.hivemq.com`:

```
dns && dns.qry.name contains "hivemq"
```

The DNS responses confirmed that `broker.hivemq.com` resolves to three IP addresses: `18.192.151.104`, `35.158.34.213`, and `35.158.43.69`. Of these, only `18.192.151.104` and `35.158.43.69` appear as destinations for MQTT traffic in the capture.

2.2.2 CQ6a - Identify all clients connecting to HiveMQ

The following filter was applied to find all `CONNECT` messages directed to the HiveMQ broker:

```
mqtt.msgtype == 1 && (ip.dst == 18.192.151.104 || ip.dst == 35.158.43.69)
```

Nine `CONNECT` messages were found, corresponding to nine distinct clients:

Packet	Broker	Client ID	ID Length
43	`18.192.151.104`	`cpoepjzkhibxgjiu`	16
130	`18.192.151.104`	`dzcxnwdqef`	10
335	`18.192.151.104`	`tukvxesuhe`	10
507	`18.192.151.104`	`fcthvjikxjul`	12
553	`18.192.151.104`	`giuxfijwus`	10
622	`18.192.151.104`	`pohujuv`	7
1747	`18.192.151.104`	`atqeblw`	7
1752	`18.192.151.104`	`rolsmcwhshlbc`	13
19	`35.158.43.69`	`pcbtjyj`	7

Table 15: Clients connecting to the HiveMQ public broker

2.2.3 CQ6a - Identify clients that erased retained values

In MQTT, a retained value is erased by publishing a message with **Retain flag = 1** and an **empty payload (0 bytes)** to the relevant topic. This overwrites the previously retained message with nothing, effectively deleting it from the broker.
To identify which clients sent such messages, the **Follow TCP Stream** function in Wireshark was used on the `CONNECT` packet of each client. Within each stream, `PUBLISH` messages with an empty payload are recognizable because the topic name is not followed by any JSON content, in contrast to normal `PUBLISH` messages which always carry a JSON payload in this capture.
The inspection was performed for all nine clients with the following results:

Client ID	Sends empty retained PUBLISH?
`cpoepjzkhibxgjiu`	× No, only subscribes
`dzcxnwdqef`	× No, all `PUBLISH` messages carry a payload
`tukvxesuhe`	× No, all `PUBLISH` messages carry a payload
`fcthvjikxjul`	× No, only subscribes
`giuxfijwus`	✓ Yes, multiple topics with empty payload found
`atqeblw`	✓ Yes, multiple topics with empty payload found
`rolsmcwhshlbc`	✓ Yes, topics with empty payload found
`pcbtjyj`	✓ Yes, multiple topics with empty payload found
`pohujuv`	✓ Yes, multiple topics with empty payload found

Table 16: Inspection results for retained value erasure

The five clients that erased retained values are: `giuxfijwus`, `atqeblw`, `rolsmcwhshlbc`, `pcbtjyj` and `pohujuv`.

2.2.4 CQ6b - Count clients with Client ID strictly longer than 7 bytes

Among the five clients identified in CQ6a, the length of each Client ID was measured:

Client ID	Length	Strictly > 7 bytes?
`giuxfijwus`	10	✓ Yes
`atqeblw`	7	× No (equal to 7, not strictly greater)
`rolsmcwhshlbc`	13	✓ Yes
`pcbtjyj`	7	× No
`pohujuv`	7	× No

Table 17: Client ID lengths among the five clients that erased retained values

Only `giuxfijwus` (10 bytes) and `rolsmcwhshlbc` (13 bytes) have a Client ID strictly longer than 7 bytes.

Answer CQ6

CQ6a = 5
Five distinct MQTT clients erased at least one retained value on the HiveMQ public broker: `giuxfijwus`, `atqeblw`, `rolsmcwhshlbc`, `pcbtjyj` and `pohujuv`.
CQ6b = 2
Two of those five clients connected to HiveMQ with a Client ID strictly longer than 7 bytes: `giuxfijwus` (10 bytes) and `rolsmcwhshlbc` (13 bytes).

2.3 CQ7

2.3.1 Identify the local broker

From the previous analysis, the local broker in `B.pcapng` is `::1` (IPv6 loopback address).

2.3.2 Filter all SUBSCRIBE requests directed to the local broker

The following Wireshark filter was applied to isolate all `SUBSCRIBE` messages sent to the local broker:

```
mqtt.msgtype == 8 && ipv6.dst == ::1
```

This returned a list of all subscription requests directed to `::1`.

2.3.3 Identify subscriptions with at least 2 wildcards

In MQTT, two wildcard characters exist:

- `+` (single-level wildcard): matches exactly one topic level.
- `#` (multi-level wildcard): matches all remaining topic levels and must always be the last character.

A topic filter was counted as valid for CQ7 if it contained **at least 2 wildcard characters** in total, counting both `+` and `#` occurrences. Topics with only 1 wildcard (such as `university/#`, `factory/fxlugs/+`, `hospital/+ /section0/temperature`) were excluded.

2.3.4 Results

The following 9 `SUBSCRIBE` requests to the local broker `::1` contain at least 2 wildcards in their topic filter:

Packet	Topic Filter	Wildcards	Count
226	`university/+/+/deposit`	`+ +`	2
363	`hospital/+/+/light`	`+ +`	2
460	`metaverse/+/room1/#`	`+ #`	2
473	`house/department3/+/+`	`+ +`	2
700	`metaverse/facility0/+/+`	`+ +`	2
1177	`hospital/fxlugs/+/#`	`+ #`	2
1915	`metaverse/+/+/hydraulic_valve`	`+ +`	2
2360	`factory/+/#`	`+ #`	2
2994	`house/+/room1/#`	`+ #`	2

Table 18: SUBSCRIBE requests to `::1` with at least 2 wildcards

All other SUBSCRIBE requests to ::1 contained either no wildcards or only a single wildcard and were therefore excluded.

Answer CQ7

CQ7 = 9
Nine SUBSCRIBE requests were sent to the local broker (::1) with a topic filter containing at least 2 wildcards.

2.4 CQ8

2.4.1 Methodology

To answer CQ8, a Python script was developed to analyze the MQTT PUBLISH messages directed to the local broker in both capture files. The script uses the pyshark library to parse the pcapng files and matplotlib to generate the histogram.

2.4.2 Identify the local brokers

From the previous analysis:

- A.pcapng: local broker is 127.0.0.1 (IPv4 loopback) on port 1883
- B.pcapng: local broker is ::1 (IPv6 loopback) on port 1883

2.4.3 Filter PUBLISH messages

The following display filters were applied to isolate only the PUBLISH messages directed to the local broker in each file:
For A.pcapng:

```
mqtt.msgtype == 3 && tcp.dstport == 1883 && ip.dst == 127.0.0.1
```

For B.pcapng:

```
mqtt.msgtype == 3 && tcp.dstport == 1883 && ipv6.dst == ::1
```

The filter tcp.dstport == 1883 ensures that only messages actually directed to the broker are considered, excluding any outgoing traffic from the broker. The IP destination filter further restricts the analysis to the local broker only, excluding any external brokers present in the captures.

2.4.4 Compute topic depth

For each PUBLISH message, the topic depth was computed by counting the number of / separators plus one:

```
def get_topic_depth(topic):
    return len(topic.split('/'))
```

For example:

- hospital/room2 → depth 2
- factory/facility2/floor1 → depth 3
- hospital/department5/room4/temperature → depth 4

2.4.5 Generate histogram

The counts per depth level were collected using Python's Counter class and plotted as a grouped bar chart with A.pcapng in blue and B.pcapng in orange.

2.4.6 Results

A.pcapng (local broker 127.0.0.1):

Topic Depth	Number of PUBLISH messages
2	154
3	131
4	154
Total (CQ8a)	**439**

Table 19: Topic depth distribution for A.pcapng

B.pcapng (local broker ::1):

Topic Depth	Number of PUBLISH messages
1	21
2	172
3	162
4	179
Total (CQ8b)	**534**

Table 20: Topic depth distribution for B.pcapng

The resulting histogram is shown in Figure 1.

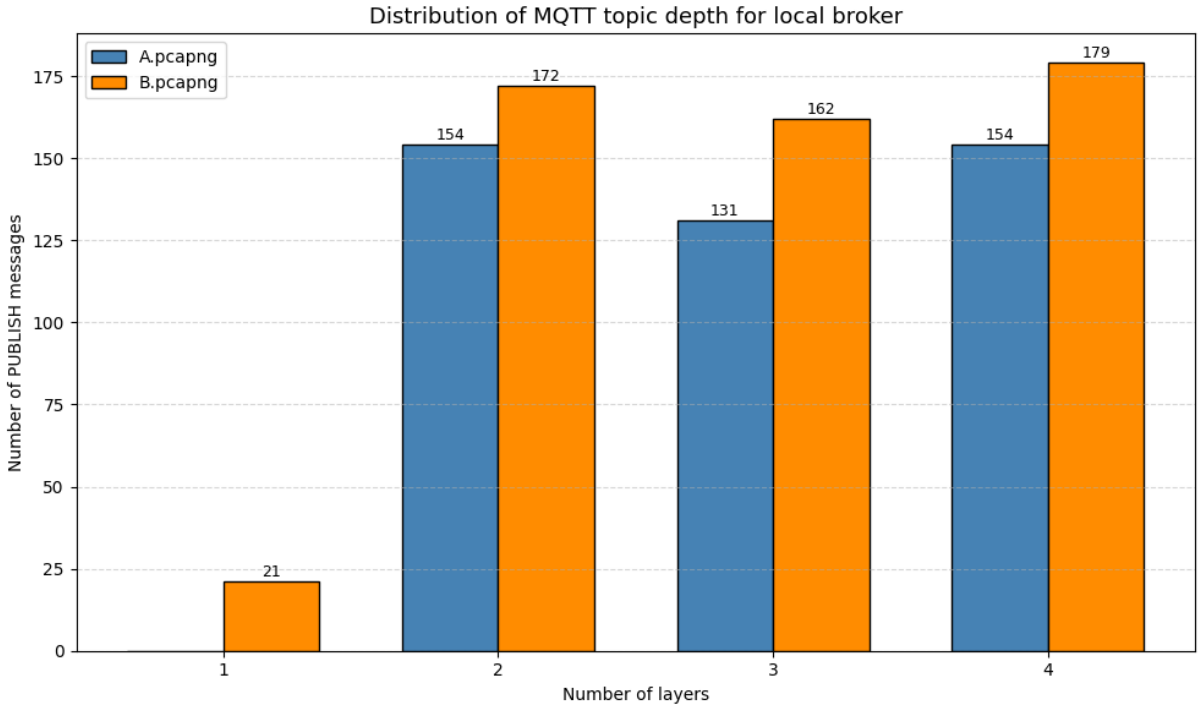


Figure 1: Distribution of MQTT topic depth for the local broker in `A.pcapng` and `B.pcapng`.

Answer CQ8

CQ8a = 439 total `PUBLISH` messages directed to the local broker in `A.pcapng`.
CQ8b = 534 total `PUBLISH` messages directed to the local broker in `B.pcapng`.